

DSA PRACTICE

One LeetCode-style problem for every Java source file in the uploaded repository

64 files → 64 questions • Questions are grouped continuously so multiple questions fit on each page. Page breaks occur naturally when the page is full.

01. Average of Three Numbers

File: Average.java Difficulty: Easy

Problem: Given three integers a, b, and c, return their integer average. Integer division should be used, so any fractional part is discarded.

Examples:

Example: a=10, b=20, c=25 → 18

Example: a=1, b=2, c=2 → 1

02. Contains Duplicate

File: HW/arrays.java Difficulty: Easy

Problem: Given an integer array, determine whether any value appears at least twice. Return true if a duplicate exists; otherwise return false.

Examples:

Example: [1,2,3,1] → true

Example: [1,2,3,4] → false

03. Average or Parity Check

File: HW/avgmethod.java Difficulty: Easy

Problem: Write a function that, for an integer n, returns whether n is even. Also implement a helper that returns the integer average of three numbers.

Examples:

Example: n=8 → true

Example: n=7 → false

04. Factorial

File: HW/factorial.java Difficulty: Easy

Problem: Given a non-negative integer n, return n!. Compute it iteratively.

Examples:

Example: n=5 → 120

Example: n=0 → 1

05. Matrix Utilities

File: HW/hwtdarrays.java Difficulty: Easy

Problem: Given a rectangular matrix, implement three operations: count occurrences of a key, compute the sum of the second row, and return the transpose of the matrix.

Examples:

Example matrix [[1,2,3],[4,5,6]]; key=5 → count=1, second-row sum=15, transpose=[[1,4],[2,5],[3,6]]

06. Leap Year and Temperature

File: HW/iffelse.java Difficulty: Easy

Problem: Given a year, determine whether it is a leap year using the Gregorian rules. Also classify a temperature as high when it is above 100.

Examples:

Example: year=2000 → leap year

Example: year=1900 → not a leap year

07. Recursive Occurrences and Number Words

File: HW/keyoccurrences.java **Difficulty:** Medium

Problem: Given an array and a key, recursively print/collect every index where the key occurs and return the total count. Extend the same recursive idea to convert a non-negative integer into its English digit names.

Examples:

Example: [3,2,4,2,2], key=2 → indices [1,3,4], count=3

08. Palindrome Number

File: HW/palindrome.java **Difficulty:** Easy

Problem: Given an integer n, determine whether its digits read the same forward and backward. Do not convert the number to a string.

Examples:

Example: 121 → true

Example: 123 → false

09. Sum of Even and Odd Inputs

File: HW/sumevenodd.java **Difficulty:** Easy

Problem: Read a sequence of integers until the user chooses to stop. Return two values: the sum of all even numbers and the sum of all odd numbers.

Examples:

Example: inputs [2,5,8,3] → evenSum=10, oddSum=8

10. Sum of Digits

File: HW/sumofdigits.java **Difficulty:** Easy

Problem: Given a non-negative integer n, return the sum of all its decimal digits.

Examples:

Example: 194 → 14

Example: 5003 → 8

11. Valid Anagram

File: Strings/hw1.java **Difficulty:** Easy

Problem: Given two strings, determine whether they are anagrams of each other. Ignore no characters: spaces and case are significant unless explicitly normalized by your implementation.

Examples:

Example: "listen", "silent" → true

Example: "rat", "car" → false

12. Shortest Path on a Grid

File: Strings/shortest.java **Difficulty:** Easy

Problem: A string contains moves N, S, E, and W, each moving one unit on a 2D grid. Starting at (0,0), return the Euclidean distance from the origin after all moves.

Examples:

Example: "WNEENESENNN" → compute the final distance

Example: "NS" → 0.0

13. Run-Length String Compression

File: Strings/strinbuilder.java **Difficulty:** Easy

Problem: Compress a string by replacing each consecutive run of the same character with the character followed by its count when the count is greater than one.

Examples:

Example: "aaabbc" → "a3b2c"

Example: "abcd" → "abcd"

14. Valid Palindrome String

File: Strings/strings.java **Difficulty:** Easy

Problem: Given a string, determine whether it is a palindrome using two pointers. Return true if it reads identically from both ends.

Examples:

Example: "racecar" → true

Example: "hello" → false

15. Area of a Square

File: areasquare.java **Difficulty:** Easy

Problem: Given the side length of a square, return its area.

Examples:

Example: side=7 → 49

16. Maximum Subarray Sum

File: arrays/allarray.java **Difficulty:** Medium

Problem: Given an integer array, return the maximum possible sum of a contiguous subarray. Solve it using Kadane's algorithm.

Examples:

Example: [-2,3,4,-5,6] → 8

Example: [-5,-2,-7] → -2

17. Binary Search

File: arrays/binarysearch.java **Difficulty:** Easy

Problem: Given a sorted array of distinct integers and a target, return the target's index using binary search, or -1 if it is absent.

Examples:

Example: [2,4,6,8,10], target=8 → 3

Example: [2,4,6], target=5 → -1

18. Best Time to Buy and Sell Stock

File: arrays/buysellstock.java **Difficulty:** Easy

Problem: Given daily stock prices, choose one day to buy and a later day to sell. Return the maximum possible profit. If no profit is possible, return 0.

Examples:

Example: [7,1,5,3,6,4] → 5

Example: [7,6,4,3,1] → 0

19. Find Maximum Element

File: arrays/largest.java **Difficulty:** Easy

Problem: Given an integer array, return the largest element without using a built-in maximum function.

Examples:

Example: [1,2,9,4,5] → 9

20. Read and Return Array

File: arrays/practice.java **Difficulty:** Easy

Problem: Given n integers, store them in an array and return the array in the same order.

Examples:

Example: input [10,20,30] → [10,20,30]

21. Reverse Array In Place

File: arrays/reversearr.java Difficulty: Easy

Problem: Given an integer array, reverse it in place using the two-pointer technique. Use $O(1)$ extra space.

Examples:

Example: $[2,3,4,5] \rightarrow [5,4,3,2]$

22. Implement Four Sorting Algorithms

File: arrays/sorting.java Difficulty: Medium

Problem: Given an integer array, implement bubble sort, selection sort, insertion sort, and counting sort as separate methods. Each method must return the sorted array.

Examples:

Example: $[5,1,4,2,8] \rightarrow [1,2,4,5,8]$

For counting sort, state the assumptions needed for the input range.

23. Search a Row- and Column-Sorted Matrix

File: arrays/sortingmatrix.java Difficulty: Medium

Problem: Given a matrix where every row and every column is sorted in ascending order, return the position of a target using the top-right search strategy. Return $[-1,-1]$ if absent.

Examples:

Example: target=32 in $[[10,20,30,40],[15,25,35,45],[27,29,37,48],[32,33,39,50]] \rightarrow [3,0]$

24. Spiral Matrix Traversal

File: arrays/spiral.java Difficulty: Medium

Problem: Given an $m \times n$ matrix, return all elements in clockwise spiral order.

Examples:

Example: $[[1,2,3],[4,5,6],[7,8,9]] \rightarrow [1,2,3,6,9,8,7,4,5]$

25. Trapping Rain Water

File: arrays/trapingwater.java Difficulty: Hard

Problem: Given an elevation map where each bar has width 1, return how much rainwater can be trapped after raining. You may use prefix maximum arrays.

Examples:

Example: $[4,2,0,6,3,2,5] \rightarrow 9$

26. Search a 2D Matrix

File: arrays/twodarray.java Difficulty: Easy

Problem: Given a rectangular integer matrix and a target, return true if the target exists anywhere in the matrix, otherwise false.

Examples:

Example: $[[1,2],[3,4]]$, target=3 $\rightarrow$ true

Example: target=7 $\rightarrow$ false

27. Total Bill with GST

File: bill.java Difficulty: Easy

Problem: Given prices of a pencil, compass, and eraser, return the total price including 18% GST.

Examples:

Example: $[100,50,50] \rightarrow 236.0$

28. Bit Manipulation Toolkit

File: bitwise/bit.java Difficulty: Medium

Problem: Implement operations to test odd/even, read the i -th bit, set the i -th bit, clear the i -th bit, update the i -th bit, clear bits from i onward, clear a range of bits, and compute n^a using binary exponentiation.

Examples:

Example: setBit(5,1) $\rightarrow$ 7

Example: fastPower(5,3) $\rightarrow$ 125

29. Process Numbers Until a Multiple of Ten

File: breakcontinue.java Difficulty: Easy

Problem: Read integers repeatedly. Whenever a number is divisible by 10, record that it is a multiple of 10 and continue reading the next value. Design a terminating version of the loop using a sentinel value.

Examples:

Example: input 7, 20, 13, -1 $\rightarrow$ report 20 and stop at -1

30. Student Score Lookup

File: collections/Demo.java Difficulty: Easy

Problem: Given a collection of student names and integer scores, store them in a hash map and support lookup of a student's score. Return -1 when the name is absent.

Examples:

Example: {"Aditya":18,"Aryan":23}, "Aryan" $\rightarrow$ 23

31. Sum of Repeated Values

File: counter.java Difficulty: Easy

Problem: Given n , compute the sum of a value added once for each iteration from 1 through n . Rewrite the loop so that the result is mathematically correct.

Examples:

Example: $n=5$, add 1 each iteration $\rightarrow$ 5

32. Concurrent Message Printing

File: demo/threads.java Difficulty: Medium

Problem: Create two threads: one prints "hi" 101 times and the other prints "hello" 101 times. Start both threads and explain why the combined output order is not deterministic.

Examples:

Example: output may interleave hi and hello in different orders on different runs.

33. Binary to Decimal

File: functions/Binarytodecimal.java Difficulty: Easy

Problem: Given a non-negative integer whose decimal digits consist only of 0 and 1, interpret those digits as a binary number and return its decimal value.

Examples:

Example: 1011 $\rightarrow$ 11

Example: 100 $\rightarrow$ 4

34. Decimal to Binary

File: functions/decimaltobinary.java Difficulty: Easy

Problem: Given a non-negative decimal integer n , return its binary representation as an integer containing only 0 and 1 digits. Handle $n=0$.

Examples:

Example: 13 $\rightarrow$ 1101

Example: 8 $\rightarrow$ 1000

35. Compute nCr

File: functions/factorialfunc.java **Difficulty:** Medium

Problem: Given n and r, return the number of ways to choose r items from n items. Implement factorial recursively and use it to compute the combination.

Examples:

Example: n=5, r=2 → 10

Example: n=6, r=3 → 20

36. Add Two Integers with a Function

File: functions/functions.java **Difficulty:** Easy

Problem: Implement a function that receives two integers and returns their sum.

Examples:

Example: 12, 7 → 19

37. Prime Number Test

File: functions/overloading.java **Difficulty:** Easy

Problem: Given an integer n, return true if n is prime and false otherwise. Check divisors only up to sqrt(n).

Examples:

Example: 29 → true

Example: 21 → false

38. Swap Two Values

File: functions/swapping.java **Difficulty:** Easy

Problem: Implement a function that swaps two integer values conceptually and explain Java's pass-by-value behavior. Then implement a method that returns the product of two integers.

Examples:

Example: multiply(2,3) → 6

39. Income Tax Slab

File: incometaxcal.java **Difficulty:** Easy

Problem: Given annual income, calculate tax using these simplified slabs: income below 500000 → 0%; 500000–700000 → 20%; above 700000 → 30%. Return the tax-inclusive amount.

Examples:

Example: income=700000 → 840000

Example: income=800000 → 1040000

40. Largest of Three Numbers

File: largestofthree.java **Difficulty:** Easy

Problem: Given three integers, return the largest value using conditional logic.

Examples:

Example: 4, 9, 2 → 9

Example: -1, -5, -3 → -1

41. Largest of Two Numbers

File: largestoftonos.java **Difficulty:** Easy

Problem: Given two integers, return the larger one.

Examples:

Example: 1,3 → 3

42. Odd or Even

File: oddeven.java Difficulty: Easy

Problem: Given an integer n, return "even" when n is divisible by 2 and "odd" otherwise.

Examples:

Example: 8 → "even"

Example: 11 → "odd"

43. Deep Copy of Student

File: oops/Ooops.java Difficulty: Medium

Problem: Design a Student class with name, age, password, and a three-element marks array. Implement a copy constructor that performs a deep copy so that changes to the original student's marks do not affect the copied student.

Examples:

Example: original marks=[100,90,80]; change original[0] to 30 → copied marks remain [100,90,80]

44. Abstract Animal Hierarchy

File: oops/abs.java Difficulty: Medium

Problem: Create an abstract Animal class with a color field, an eat() method, and an abstract walk() method. Implement Horse and Chicken so that each provides its own walking behavior.

Examples:

Example: Horse.walk() → "walks on four legs"; Chicken.walk() → "walks on two legs"

45. Multiple Behaviors with Interfaces

File: oops/interfaces.java Difficulty: Medium

Problem: Define Herbivore and Carnivore interfaces and implement both in an Omnivore class. The class must provide eatGrass() and eatAnimal() behaviors.

Examples:

Example: Omnivore can perform both eating behaviors.

46. Multilevel Inheritance

File: oops/object.java Difficulty: Easy

Problem: Create an Animal base class with eat() and breath(), a Mammal subclass with legs, and a Dog subclass with breed. Demonstrate that Dog objects inherit members from both ancestors.

Examples:

Example: Dog with legs=4 should expose legs and inherited eat()/breath() methods.

47. Polymorphic Animal and Calculator

File: oops/poly.java Difficulty: Medium

Problem: Implement method overloading in Calculator for adding two ints, two floats, and three ints. Also implement method overriding so Deer provides its own eat() behavior.

Examples:

Example: sum(1,2) → 3; sum(1.5f,2.5f) → 4.0; Deer.eat() → "eat grass"

48. Static School and Constructor Inheritance

File: oops/stat.java Difficulty: Easy

Problem: Create a Student class with a static school field shared by all students. Create Animal and Horse classes where Horse's constructor initializes an inherited color field.

Examples:

Example: changing Student.school through one object should be visible through another object.

49. Generate Classic Console Patterns

File: patterns/allpattern.java Difficulty: Medium

Problem: Given n, implement functions that print a star triangle, number pattern, Floyd's triangle, butterfly, 0-1 triangle, rhombus, hollow rhombus, and diamond pattern.

Examples:

Example for Floyd's triangle n=4:

```
1
2 3
4 5 6
7 8 9 10
```

50. Character Pyramid

File: patterns/characterpyramid.java Difficulty: Easy

Problem: Given n, print a left-aligned character pyramid beginning with A and advancing to the next character after every printed position.

Examples:

Example n=3:

```
A
BC
DEF
```

51. Number Half Pyramid

File: patterns/halfpyramid.java Difficulty: Easy

Problem: Given n, print n rows where row i contains the integers 1 through i without spaces.

Examples:

Example n=4:

```
1
12
123
1234
```

52. Hollow Rectangle

File: patterns/hollowrectangle.java Difficulty: Easy

Problem: Given rows and columns, print a rectangle whose border is made of '*' and whose interior is spaces.

Examples:

Example 4x5:

```
*****
* *
* *
* *
*****
```

53. Inverted Star Pyramid

File: patterns/invertedpyramid.java Difficulty: Easy

Problem: Given n, print an inverted left-aligned star pyramid with n stars in the first row and one star in the last row.

Examples:

Example n=4:

```
****
***
**
*
```


54. Star and Inverted Star Patterns

File: patterns/starpattern.java Difficulty: Easy

Problem: Given n, print both a normal increasing star pattern and an inverted decreasing star pattern.

Examples:

Example n=4 normal:

```
*
**
***
****
```

55. Print All Primes Below N

File: primeno.java Difficulty: Easy

Problem: Given n, return all prime numbers strictly less than n.

Examples:

Example: n=10 → [2,3,5,7]

Example: n=2 → []

56. Sort an Array with Divide and Conquer

File: recursion/divideandconquer.java Difficulty: Medium

Problem: Given an integer array, sort it in ascending order using merge sort or quicksort implemented recursively. Your solution should perform all changes in the same array.

Examples:

Example: [6,3,9,8,2,5] → [2,3,5,6,8,9]

57. Friends Pairing

File: recursion/friends.java Difficulty: Medium

Problem: There are n friends. Each friend may remain single or pair with exactly one other friend. Return the number of possible arrangements. Also explain how to generate binary strings of length n without consecutive 1s.

Examples:

Example: n=3 → 4

Example valid binary strings for n=3 include 000,001,010,100,101

58. Recursive Fundamentals

File: recursion/ntoone.java Difficulty: Medium

Problem: Implement recursive functions to print 1..n, compute n!, compute the sum 1..n, and return the nth Fibonacci number. Use correct base cases.

Examples:

Example: factorial(5) → 120

sum(5) → 15

fib(6) → 8

59. Recursive Array and Power Utilities

File: recursion/sorted.java Difficulty: Medium

Problem: Implement recursive checks for whether an array is sorted, first occurrence of a key, last occurrence of a key, x^n , and optimized x^n using exponentiation by squaring.

Examples:

Example: [1,2,2,4], key=2 → first=1, last=2

power(2,10) → 1024

60. Recursive-Style Array Transformation

File: recursion/testing.java Difficulty: Easy

Problem: Given an integer array, for every index i before the last, replace $\text{arr}[i]$ with $\text{arr}[i+1]+1$. Return the transformed array.

Examples:

Example: $[1,2,3,4] \rightarrow [3,4,5,4]$

61. Tiling a $2 \times n$ Board

File: recursion/tiling.java Difficulty: Medium

Problem: Given a $2 \times n$ board and 2×1 tiles, return the number of ways to tile the board. Also implement recursive duplicate removal from a lowercase string using a boolean map.

Examples:

Example: $n=4 \rightarrow 5$

Example: "apnnacollege" $\rightarrow$ "apncollege"

62. Reverse an Integer

File: reverseofno.java Difficulty: Easy

Problem: Given an integer n , return its digits reversed. Do not use string reversal.

Examples:

Example: 12340 $\rightarrow$ 4321

Example: 507 $\rightarrow$ 705

63. Solid Square Pattern

File: squarepattern.java Difficulty: Easy

Problem: Given n , print an $n \times n$ square made entirely of '*'.

Examples:

Example $n=3$:

64. Pass or Fail

File: ternaryop.java Difficulty: Easy

Problem: Given marks, return "PASS" when marks are at least 60 and "FAIL" otherwise. Implement the decision using a ternary operator.

Examples:

Example: 75 $\rightarrow$ "PASS"

Example: 50 $\rightarrow$ "FAIL"